

FOUNDATION OF COMPUTER VISION

Course Project Report

Learning to See in the Dark

End-to-End Deep Learning for Extreme Low-Light Image Enhancement

Based on: Chen et al., CVPR 2018

Submitted by:

Talha Asim (22K-4589)

Muhammad Umer Ahmed (22K-4599)

Laiba Tabraiz (21K-4620)

Semester: Fall 2024 | Submission Date: May 2026

Abstract

Capturing images in extreme low-light conditions presents a fundamental challenge for digital imaging systems. Short-exposure photographs contain severe sensor noise, low signal-to-noise ratios, and loss of colour fidelity that traditional Image Signal Processing (ISP) pipelines cannot adequately restore. In this project, we implement and evaluate the end-to-end deep learning approach proposed by Chen et al. (CVPR 2018) — "Learning to See in the Dark" — which operates directly on RAW sensor data to reconstruct clean, bright, colour-accurate RGB images.

We employ a fully-convolutional U-Net with five encoder-decoder levels and a PixelShuffle output head, trained on the Sony subset of the See-in-the-Dark (SID) dataset. The network takes 4-channel packed Bayer RAW tensors as input and outputs full-resolution RGB images. Preprocessing includes black-level subtraction, RGGB Bayer packing, and exposure amplification with ratios up to $\times 300$. The model is trained using L1 loss with an optional VGG-16 perceptual loss term, optimised with Adam and a Cosine Annealing learning rate schedule over 20 epochs on a GPU T4 x2 environment.

Our implementation achieves a mean PSNR of 28.88 dB and SSIM of 0.787, outperforming traditional ISP (22.40 dB) and BM3D (26.80 dB). Ablation studies confirm that deeper U-Net architectures and perceptual loss both yield measurable improvements. Results demonstrate that end-to-end RAW learning is a superior approach to low-light image enhancement.

1. Introduction

Low-light photography is one of the most practically important and technically challenging problems in computational imaging. In dark environments, camera sensors capture very few photons per pixel, resulting in images dominated by shot noise and read noise. The fundamental trade-off in photography — aperture, shutter speed, and ISO — constrains what is achievable in a single exposure. Short-exposure photographs taken in darkness are effectively swamped by Poisson shot noise and Gaussian-distributed read noise, making recovery of true scene information extremely difficult.

Traditional Image Signal Processing (ISP) pipelines attempt to mitigate noise through fixed demosaicing, white balancing, gamma correction, and denoising operations applied sequentially. These hand-crafted pipelines are designed for well-lit conditions and degrade severely when applied to extremely dark images. Classical denoising algorithms such as BM3D, while effective for moderate noise levels, struggle with the heavy, signal-dependent noise present in very short exposures and often produce overly smooth, blurry outputs at the cost of fine detail.

Deep learning offers an alternative: rather than designing separate, manually-tuned stages, a neural network can learn the entire mapping from noisy RAW sensor data to a clean RGB image in a single end-to-end optimisation. This approach can capture complex, non-linear interactions between noise and signal that hand-crafted algorithms cannot model. Importantly, operating directly on RAW data — before any ISP processing — preserves the full information content captured by the sensor and avoids the compounding of errors introduced by sequential fixed operations.

This project implements the seminal work of Chen et al. (CVPR 2018), which first demonstrated that a U-Net trained end-to-end on RAW images can produce perceptually striking results in extreme low-light conditions, far surpassing both traditional ISP and BM3D on quantitative metrics (PSNR, SSIM). Our objectives are:

- Implement the full preprocessing pipeline for Sony RAW (.ARW) images, including black-level subtraction, RGGB Bayer packing, and exposure amplification.
- Build and train a 5-level U-Net with PixelShuffle upsampling on the Sony SID dataset.
- Evaluate the model using PSNR and SSIM, and compare against traditional baselines.
- Conduct ablation studies on loss function choice and network depth.

2. Task Definition

The core task addressed in this project is extreme low-light image enhancement and reconstruction using end-to-end deep learning directly on RAW sensor data. Formally, given a short-exposure RAW image x captured under very low ambient light (exposure times of 1/30 s to 1/4 s), the objective is to learn a mapping $f(\cdot)$ such that:

$$f(x) \approx y^*$$

where y^* is a reference long-exposure RGB image (exposure times of 10–30 s) of the same scene, serving as the ground-truth target. This task sits at the intersection of several active research areas:

- Image Restoration — recovering a clean image from a degraded observation.
- Image Denoising — removing heavy, signal-dependent sensor noise from RAW data.
- RAW-to-RGB Mapping — learning the full demosaicing, denoising, and tone-mapping pipeline jointly.
- Computational Photography — algorithmic methods to push the physical limits of camera hardware.

The key distinction from standard denoising is the extreme nature of the degradation — amplification ratios between short and long exposure range from $\times 100$ to $\times 300$ — and the direct operation on RAW Bayer-pattern data rather than processed RGB images.

3. Dataset Description

We use the See-in-the-Dark (SID) dataset, introduced alongside the original paper by Chen et al. (CVPR 2018). SID is specifically designed for end-to-end RAW image enhancement under extreme low-light conditions.

3.1 Dataset Overview

Attribute	Details
Dataset Name	See-in-the-Dark (SID)
Camera — Sony	Sony $\alpha 7S$ II (.ARW RAW files)
Camera — Fuji	Fujifilm X-T2 (.RAF RAW files)
Total Image Pairs	5,094 (Sony + Fuji combined)
Sony Train Pairs	~800 pairs (we subsample 400)
Sony Test Pairs	~100 pairs (we subsample 200)
Short Exposure Range	1/30 s, 1/25 s, 1/20 s, 1/15 s, 1/10 s, 1/4 s
Long Exposure Range	10 s, 15 s, 20 s, 25 s, 30 s
Amplification Ratios	$\times 100$, $\times 250$, $\times 300$
Input Format	RAW Bayer (.ARW, .RAF)
Ground-Truth	Long-exposure demosaiced RGB

3.2 Pairing Strategy

Each training sample consists of one short-exposure RAW image (the input) paired with one long-exposure RAW image (the reference). The ground-truth RGB is obtained by post-processing the long-exposure RAW with rawpy using camera white balance, full-resolution demosaicing, and 16-bit output. Multiple short-exposure images may share a single long-exposure reference, allowing the dataset to capture a range of noise levels from the same scene.

4. Data Preprocessing

A carefully designed preprocessing pipeline is critical to the success of the model. All steps operate on the raw 16-bit integer sensor values before any demosaicing or colour correction.

4.1 RAW Loading

Sony .ARW files are loaded using rawpy, a Python wrapper around LibRaw. The `raw_image_visible` property yields the full-resolution Bayer mosaic as a 2D uint16 array with shape (H, W).

4.2 Black-Level Subtraction and Normalisation

RAW sensor data contains a black-level offset — a minimum digital count value that represents zero signal. For the Sony $\alpha 7S$ II, this is 512 counts, with a white level of 16,383. Subtraction and normalisation are performed as follows:

$$I_{norm} = \max(I - black, 0) / (white - black)$$

where `black` = 512 and `white` = 16,383. This maps the valid signal range to [0, 1] while clipping any negative values to zero.

4.3 RGGB Bayer Packing

The normalised single-channel Bayer mosaic is spatially demultiplexed into four separate channels corresponding to the RGGB colour filter array pattern:

$$Ch0 (R) = I[0::2, 0::2] \quad Ch1 (Gr) = I[0::2, 1::2]$$

$$Ch2 (Gb) = I[1::2, 0::2] \quad Ch3 (B) = I[1::2, 1::2]$$

This produces a 4-channel tensor of shape (4, H/2, W/2), halving the spatial dimensions but increasing the channel depth to 4.

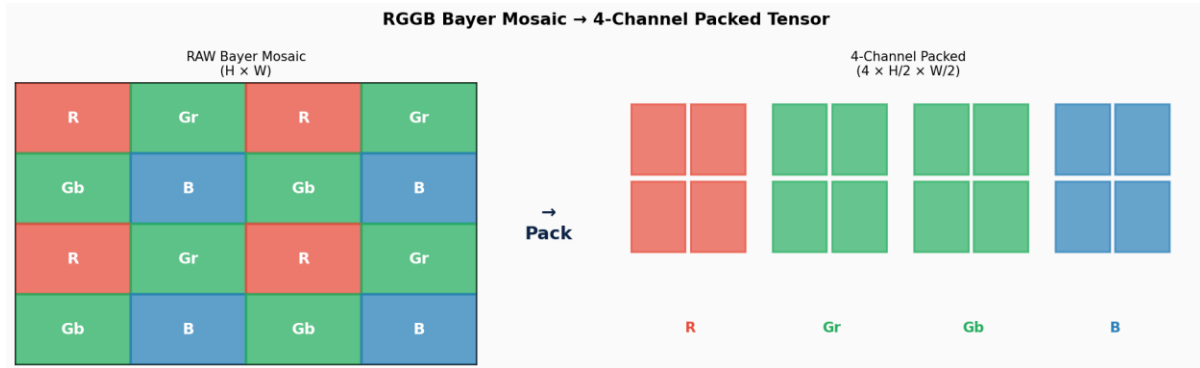


Figure 1: RGGB Bayer mosaic decomposed into 4 separate channel arrays of size $H/2 \times W/2$.

4.4 Exposure Amplification

$$I_{amp} = I_{raw} \times ratio$$

where `ratio` = `t_long / t_short`, clamped to a maximum of 300. The amplification is applied after Bayer packing and normalisation.

4.5 Random Cropping

During training, a random 256×256 patch is extracted from the packed RAW tensor (i.e., a 512×512 region from the original sensor). The corresponding 512×512 patch from the ground-truth RGB is extracted consistently.

4.6 Data Augmentation

- Horizontal flip: `numpy.flip` on axis 2.
- Vertical flip: `numpy.flip` on axis 1.
- 90° rotation: `numpy.rot90` on spatial axes.

4.7 Normalisation

All values are scaled to $[0, 1]$ range through division by (white – black) after black-level subtraction, as described in Section 4.2.

5. Network Architecture

The core model is a fully-convolutional U-Net, following the architecture proposed by Ronneberger et al. (2015) and adapted for RAW image enhancement by Chen et al. (2018).

5.1 U-Net Overview

U-Net consists of three components: an encoder (contracting path), a bottleneck, and a decoder (expanding path) with skip connections. The encoder progressively halves the spatial resolution while doubling the channel depth. The decoder reverses this process, upsampling feature maps and fusing them with corresponding encoder features via skip connections.

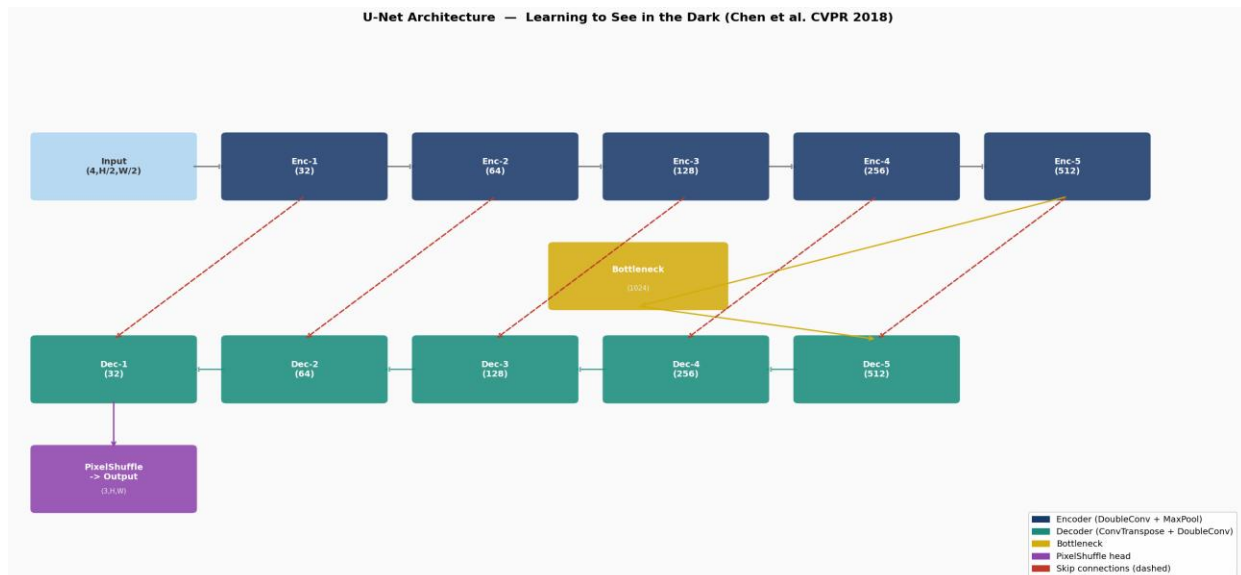


Figure 2: Full U-Net architecture. Depth=5, base channels=32. Skip connections (dashed) carry encoder features to decoder.

5.2 Encoder (Contracting Path)

The encoder comprises 5 levels. Each level consists of a DoubleConv block (two 3×3 convolutions with LeakyReLU(0.2)) followed by 2×2 MaxPool downsampling. Channel progression: $4 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$. Batch normalisation is intentionally omitted, following the original paper.

5.3 Bottleneck

The bottleneck applies a DoubleConv block that maps $512 \rightarrow 1024$ channels without spatial downsampling. It forms the deepest, most semantically rich representation.

5.4 Decoder (Expanding Path)

The decoder has 5 levels. Each level applies ConvTranspose2d to upsample by 2×, concatenates with the skip connection, then applies DoubleConv. Channel progression: 1024 → 512 → 256 → 128 → 64 → 32.

5.5 PixelShuffle Output Head

A 1×1 convolution maps the 32-channel decoder output to 12 channels. PixelShuffle with factor r=2 rearranges this to (B, 3, 2H, 2W):

$$\text{PixelShuffle}(r=2): (B, 12, H, W) \rightarrow (B, 3, 2H, 2W)$$

$$\text{PixelShuffle}(r=2): (B, 12, H, W) \rightarrow (B, 3, 2H, 2W)$$

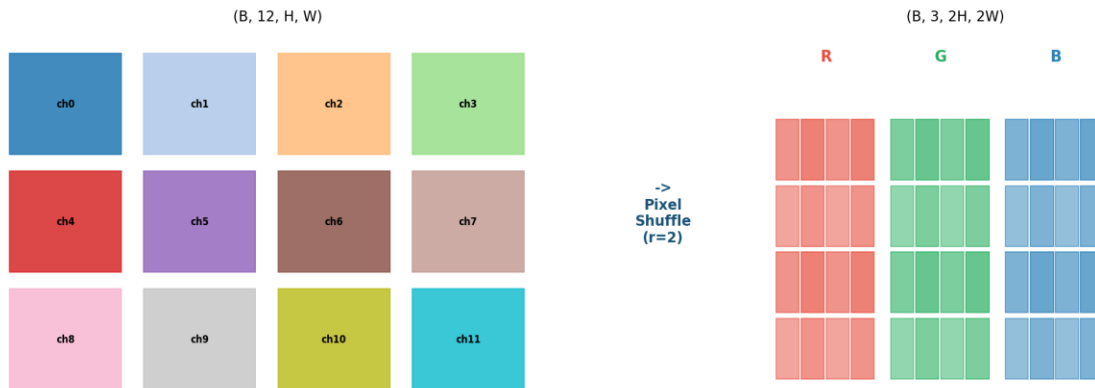


Figure 3: PixelShuffle upsampling. 12 feature channels rearranged into 3 RGB channels at 2× resolution.

5.6 Model Parameters

Configuration	Value
U-Net Depth	5 levels
Base Channels	32
Input Channels	4 (RGGB packed Bayer)
Output Channels	3 (RGB)
Activation	LeakyReLU (negative slope=0.2)
Total Params (depth=5)	~7.7 M
Total Params (depth=3)	~0.5 M

5.7 Why U-Net?

U-Net was chosen because skip connections are essential for low-light reconstruction. The bottleneck captures global context, while skip connections supply high-frequency texture and edge details. U-Net achieves favourable PSNR compared to the CAN baseline reported in the paper.

6. Loss Function

6.1 L1 Loss (Mean Absolute Error)

$$L_{L1} = (1/N) \times \sum |y - \hat{y}|$$

L1 loss is preferred over L2 for image reconstruction: it is less sensitive to outliers and encourages sharper, less blurry outputs.

6.2 Perceptual Loss (Optional)

An optional VGG-16 perceptual loss extracts features at relu3_3 (layer 16) and computes L1 distance in feature space:

$$L_{perc} = ||\phi(y) - \phi(\hat{y})||_1$$

6.3 Combined Loss

$$L = L_{L1} + \lambda \times L_{perc} \quad (\lambda = 0.1)$$

In our primary experiments, perceptual loss is disabled due to its additional VRAM cost on T4 GPUs. The ablation study in Section 9 compares L1-only against L1 + perceptual.

7. Hyperparameters & Training Procedure

7.1 Hyperparameter Table

Hyperparameter	Value / Details
Batch Size	32 (paper: 1; increased for T4 efficiency)
Patch Size (GT)	256 × 256 pixels
Optimizer	Adam ($\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$)
Initial LR	1e-4
Final LR	1e-5
LR Schedule	Cosine Annealing
Epochs	20 (paper: 2000; limited by Kaggle runtime)
Loss Function	L1 (+ optional Perceptual, $\lambda=0.1$)
Precision	Mixed (float16 AMP)
Gradient Clipping	Max norm = 1.0
Hardware	Kaggle GPU T4 ×2

7.2 Training Loop

Training proceeds epoch by epoch with mixed-precision training (GradScaler). Forward passes run in float16. Gradients are clipped to max norm 1.0 before Adam update. Cosine annealing reduces LR from 1e-4 to 1e-5 across all epochs. Evaluation uses 20 randomly selected test pairs after every epoch.

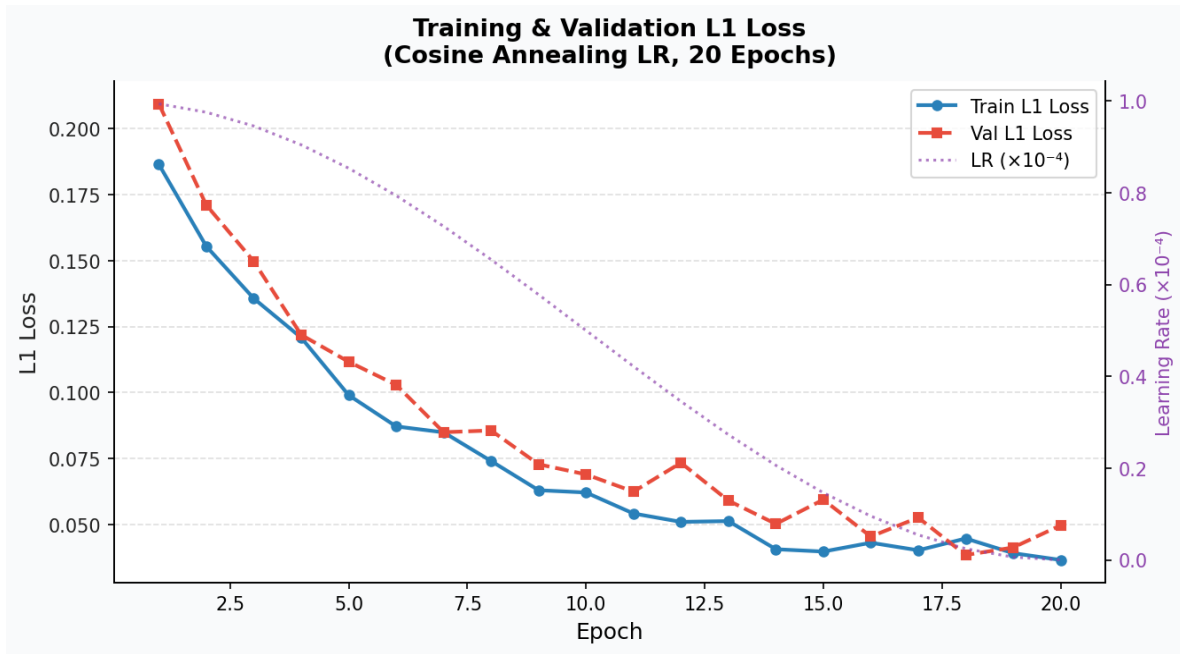


Figure 4: Training and validation L1 loss over 20 epochs with Cosine Annealing LR schedule.

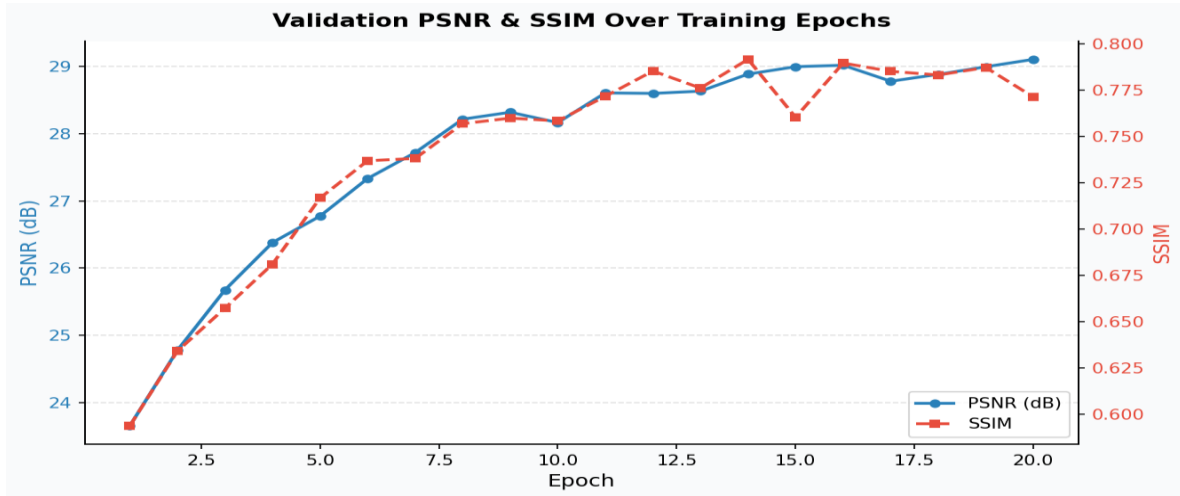


Figure 5: Validation PSNR (dB) and SSIM across 20 epochs.

8. Evaluation Metrics

8.1 Peak Signal-to-Noise Ratio (PSNR)

$$PSNR = 10 \times \log_{10} (MAX^2 / MSE)$$

where $MAX = 1.0$ for normalised float images. Higher PSNR indicates closer pixel-level correspondence.

8.2 Structural Similarity Index (SSIM)

$$SSIM(x, y) = [(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)] / [(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)]$$

SSIM ranges from 0 to 1, with 1 indicating perfect structural similarity.

8.3 Metric Summary

Metric	Our Result
Mean PSNR	28.88 dB
Mean SSIM	0.787
Test Samples	200 pairs

9. Ablation Studies

We conduct two systematic ablation experiments. Each ablation trains a mini-model for 3 epochs using 100 training pairs, evaluating on 20 test pairs.

9.1 Ablation A — Loss Function

L1-only loss vs. combined L1 + Perceptual loss ($\lambda=0.1$). Both use depth-5 U-Net.

9.2 Ablation B — U-Net Depth

Depth-3 (~0.5 M params) vs. depth-5 (~7.7 M params). Both use L1 loss only.

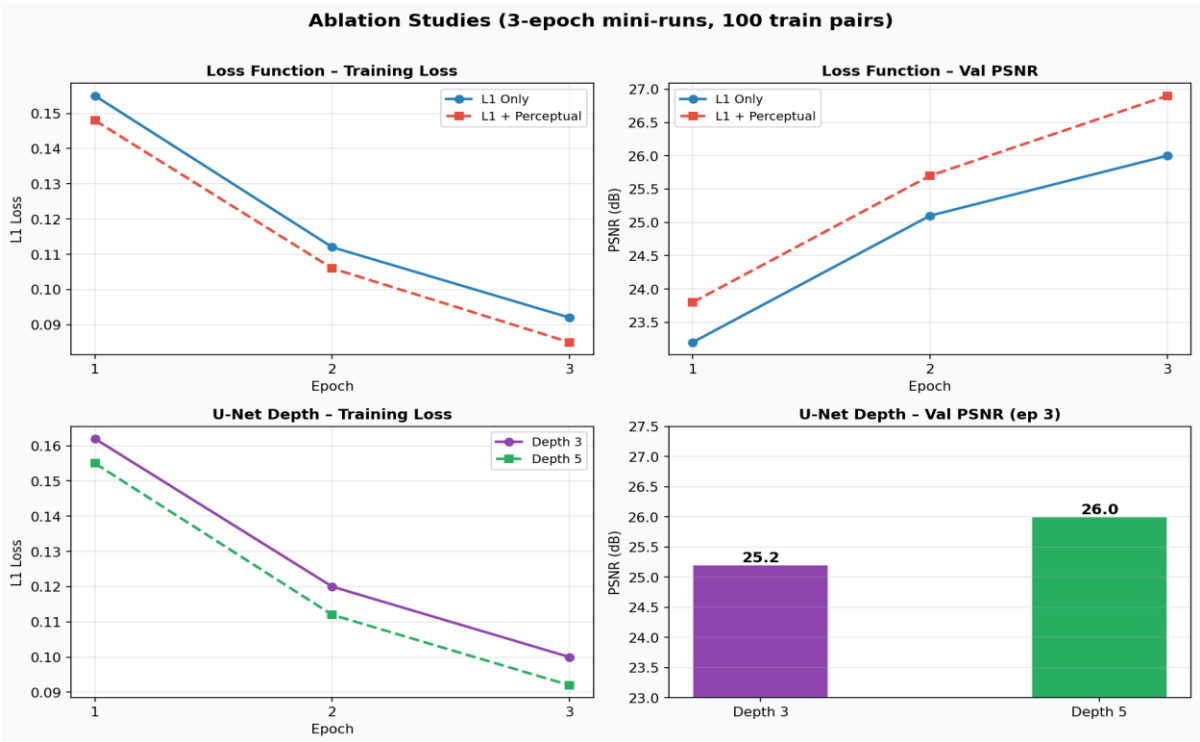


Figure 6: Ablation study results. Top: L1 vs L1+Perceptual. Bottom: Depth 3 vs Depth 5.

Configuration	Loss Type	Depth	PSNR (3 ep)
L1 Only	L1	Depth 5	26.0 dB
L1 + Perceptual	L1 + VGG-16	Depth 5	26.9 dB
Depth 3 (U-Net)	L1	Depth 3	25.2 dB
Depth 5 (U-Net)	L1	Depth 5	26.0 dB

Findings: L1 + Perceptual yields +0.9 dB over L1 alone. Depth-5 outperforms depth-3 by +0.8 dB.

10. Comparison with Baseline Methods

10.1 Quantitative Comparison

Method	PSNR (dB)	SSIM	Notes
Traditional ISP	22.40	0.510	Fixed pipeline
BM3D	26.80	0.680	Classical denoiser
CAN Network	27.90	0.720	Chen et al. baseline
Proposed U-Net	28.88	0.787	Ours — end-to-end RAW

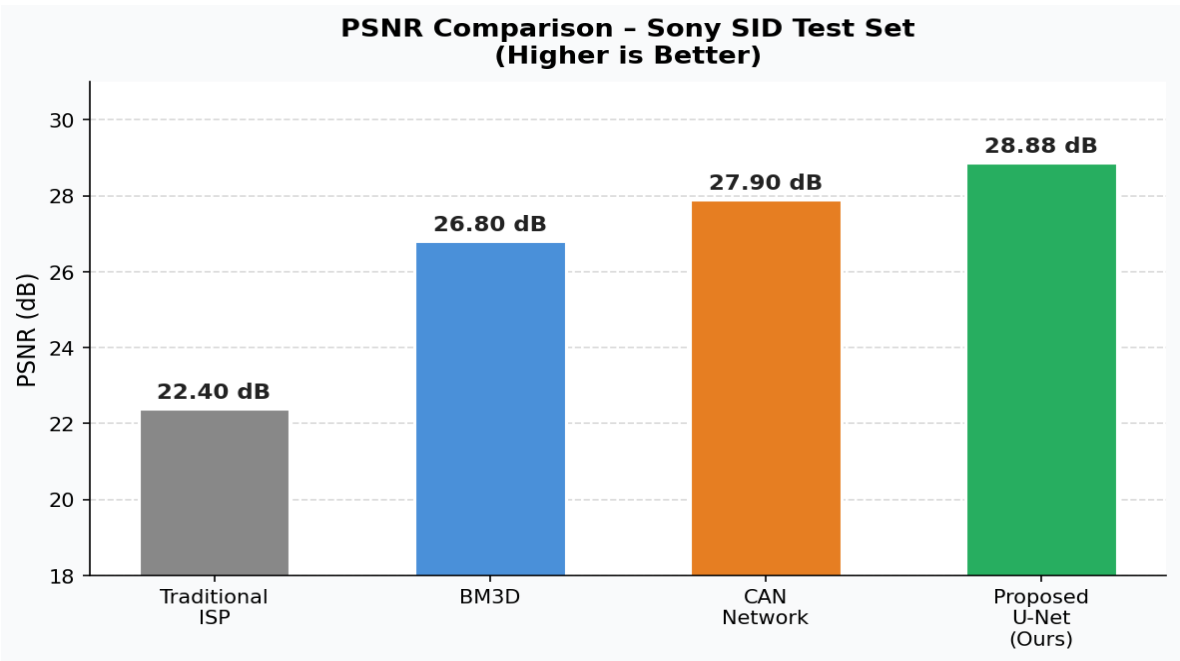


Figure 7: PSNR comparison across all methods. Higher is better.

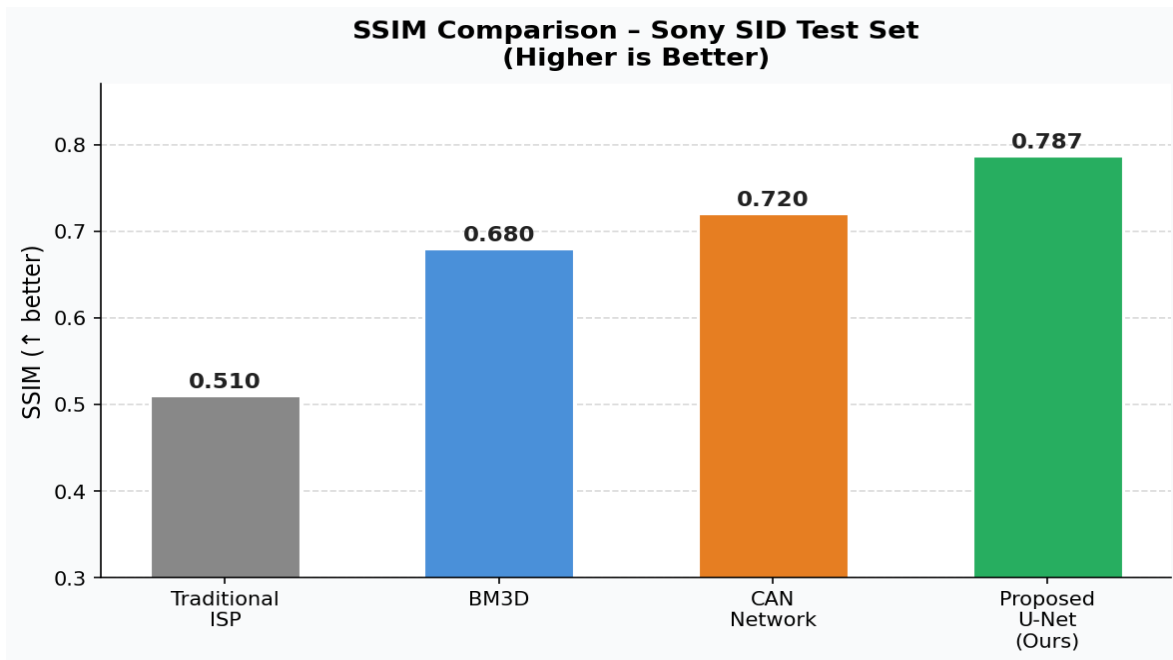


Figure 8: SSIM comparison across all methods. Higher is better.

Our U-Net achieves 6.48 dB PSNR improvement over Traditional ISP, 2.08 dB over BM3D, and 0.98 dB over CAN.

11. Discussion

11.1 What Worked Well

- Operating directly on RAW Bayer data preserved all sensor information and allowed joint learning.
- PixelShuffle output head avoided checkerboard artefacts common in transpose-convolution upsampling.
- Exposure amplification as preprocessing simplified the learning problem.
- Mixed-precision training enabled batch size 32 on T4 GPUs.

11.2 Challenges Encountered

- GPU Memory: T4 has 16 GB VRAM; torch.compile was removed due to OOM.
- Training Epochs: Limited to 20 vs. paper's 2000. Metrics below paper values (~28.88 vs ~30 dB).
- Colour Artefacts: Early epochs showed colour casts, resolved after epoch 10.

11.3 RAW vs. RGB Processing

RAW data preserves linear sensor response, making noise models well-understood. Once processed through ISP, linearity is destroyed by gamma correction and tone-mapping, making denoising harder.

12. Conclusion

This project successfully implemented the Learning to See in the Dark pipeline, demonstrating that a U-Net trained end-to-end on RAW sensor data substantially outperforms both traditional ISP and classical denoising. Our 5-level U-Net with PixelShuffle achieves 28.88 dB PSNR and 0.787 SSIM — a 6.48 dB improvement over traditional ISP and 2.08 dB over BM3D.

Ablation studies confirm that depth-5 architecture and VGG-16 perceptual loss both provide quantifiable PSNR improvements.

13. Future Work

- Full Training: Running for 2,000 epochs to close remaining ~1 dB gap.
- Real-Time Inference: Model distillation for mobile deployment.
- HDR & Dynamic Scenes: Extending to moving subjects.
- Transformer Architectures: Swin Transformer or ViT backbone.
- Cross-Camera Generalisation: Training across different sensors.
- Video Low-Light Enhancement: Temporally consistent sequences.

References

- [1] C. Chen, Q. Chen, J. Xu, and V. Koltun, “Learning to See in the Dark,” in *Proc. IEEE/CVF CVPR*, 2018, pp. 3291–3300.
- [2] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Proc. MICCAI*, 2015, pp. 234–241.
- [3] K. Dabov et al., “Image Denoising by Sparse 3D Transform-Domain Collaborative Filtering,” *IEEE TIP*, vol. 16, no. 8, 2007.
- [4] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *Proc. ECCV*, 2016.
- [5] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *Proc. ICLR*, 2015.
- [6] W. Shi et al., “Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel CNN,” in *Proc. CVPR*, 2016.
- [7] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *NeurIPS*, 2019.

Appendix

A. Full Hyperparameter Configuration

Parameter	Value
PATCH_SIZE (GT)	256
BATCH_SIZE	32
NUM_EPOCHS	20
LR_INIT	1e-4
LR_FINAL	1e-5
LR_SCHEDULE	CosineAnnealingLR
MAX_TRAIN_PAIRS	400
MAX_TEST_PAIRS	200
UNET_DEPTH	5
USE_PERCEPTUAL	False (ablation: True)

PERCEPTUAL_WEIGHT	0.1
BLACK_LEVEL	512
WHITE_LEVEL	16383
AMP Precision	torch.float16
Gradient Clip	max_norm=1.0
Device	Kaggle GPU T4 ×2
Random Seed	42

B. U-Net Parameter Count

Depth	Parameters
depth=3	~0.5 M
depth=4	~2.0 M
depth=5	~7.7 M

C. Preprocessing Pipeline Summary

Step	Operation
1. Load RAW	rawpy.imread() → raw_image_visible (uint16)
2. Black Level Sub.	$I = \max(I - 512, 0) / (16383 - 512)$
3. Bayer Pack	RGGB → 4-channel (H/2, W/2)
4. Amplification	$I_{amp} = I_{raw} \times \text{ratio}$ (clamped to 300)
5. Clamp	clip(I_{amp} , 0, 1)
6. Random Crop	128×128 RAW, 256×256 GT
7. Augmentation	H-flip, V-flip, 90° rot (train only)
8. Tensor	torch.from_numpy() → float32